

P2216R0

std::format improvements

Published Proposal, 2020-09-06

This version:

<http://fmt.dev/papers/p2216r0.html>

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Introduction
- 2 Compile-time checks
- 3 Binary size
- 4 Impact on existing code
- 5 Wording
- 6 Acknowledgements

References

Informative References

"Safety doesn't happen by accident."
— unknown

§ 1. Introduction

This paper proposes the following improvements to the C++20 formatting facility:

- Improving safety via compile-time format string checks
- Reducing binary code size of `format_to`

§ 2. Compile-time checks

Consider the following example:

```
std::string s = std::format("{:d}", "I am not a number");
```

In C++20 ([N4861]) it throws `format_error` because `d` is not a valid format specifier for a null-terminated character string.

We propose making it ill-formed so that given a proper language facility ([P1045], [P1221] or similar) this results in a compile-time rather than a runtime error. This will significantly improve safety of the formatting API and bring it on par with other languages such as D ([D-FORMAT]) and Rust ([RUST-FMT]).

This proposal has been shown to work on a version of clang that implements [P1221]: <https://godbolt.org/z/hcnxfY>. Format string parsing in C++20 has been designed with such checks in mind ([P0645]) and is already `constexpr`.

Without a language or implementation support it's only possible to emulate the desired behavior by passing format strings wrapped in a `consteval` function, a user-defined literal, a macro or as a template parameter, for example:

```
std::string s = std::format(std::static_string("{:d}"), "I am not a number"
```

This is clearly not a satisfactory solution because it doesn't improve safety of the existing API (another wrong default). Template parameters additionally introduce confusing API that interacts poorly with argument indexing.

From the extensive usage experience in the {fmt} library ([FMT]) that provides compile-time checks as an opt-in we've found that users expect errors in literal format strings to be diagnosed at compile time by default. One of the reasons is that such diagnostic is commonly done in `printf`, for example:

```
printf("%d", "I am not a number");
```

gives a warning both in GCC and clang:

```
warning: format specifies type 'int' but the argument has type 'const char'
```

so users expect the same or better level of diagnostics from a similar C++ facility.

§ 3. Binary size

The `vformat_to` functions take format args parameterized on the output iterator via the formatting context:

```
template<class Out, class charT>
    using format_args_t = basic_format_args<basic_format_context<Out, charT>

template<class Out>
    Out vformat_to(Out out, string_view fmt,
                   format_args_t<type_identity_t<Out>, char> args);
```

Unfortunately it may result in significant code bloat because formatting code will have to be instantiated for every iterator type used with `format_to` or `vformat_to`. This happens even for argument types that are not formatted, clearly violating "you don't pay for what you don't use" principle. Also this is unnecessary because the iterator type can be erased via the internal buffer as it is done in `format` and `vformat`. Therefore we propose using `format_args` and `wformat_args` instead of `format_args_t` in these overloads:

```
template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args args);
```

`formatter` specializations will continue to support output iterators so this only affects type-erased API and not the one with compiled format strings that will be proposed separately. The latter will not be affected by the code bloat issue because instantiations will be limited only to used argument types.

This proposal has been successfully implemented in the `{fmt}` library ([\[FMT\]](#)).

§ 4. Impact on existing code

Making invalid format strings ill-formed and removing problematic `vformat_to` overloads are breaking changes although at the time of writing none of the standard libraries implements the C++20 formatting facility and therefore there is no code using it.

§ 5. Wording

All wording is relative to the C++ working draft [\[N4861\]](#).

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [\[version.syn\]](#):

Change in [\[format.err.report\]](#):

Formatting functions throw `format_error` if an argument `fmt` is passed that is not a format string for `args`. They propagate exceptions thrown by operations of formatter specializations and iterators. Failure to allocate storage is reported by throwing an exception as described in [\[res.on.exception.handling\]](#).

Passing an argument `fmt` that is not a format string for parameter pack `args` is ill-formed with no diagnostic required.

Change in [\[format.syn\]](#):

```
template<class Out, class charT>  
using format_args_t = basic_format_args<basic_format_context<Out, charT>
```

<

>

...

```

template<class Out>
    Out vformat_to(Out out, string_view fmt,
                    format_args_t<type_identity_t<Out>, char> args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt,
                    format_args_t<type_identity_t<Out>, wchar_t> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                    format_args_t<type_identity_t<Out>, char> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                    format_args_t<type_identity_t<Out>, wchar_t> args);

```

```

template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt, wformat_args args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                    format_args args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                    wformat_args args);

```

Change in [\[format.functions\]](#):

```

template<class Out, class... Args>
    Out format_to(Out out, string_view fmt, const Args&... args);
template<class Out, class... Args>
    Out format_to(Out out, wstring_view fmt, const Args&... args);

```

Effects: Equivalent to:

```

using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(out, fmt, make_format_args<context>(args...));

```

```

return vformat_to(out, fmt, make_format_args(args...));

```

```

template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, string_view fmt, const Args&...)
template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, wstring_view fmt, const Args&...)

```

Effects: Equivalent to:

```

using context = basic_format_context<Out, decltype(fmt)::value_type>;
return vformat_to(out, loc, fmt, make_format_args<context>(args...));

```

```

return vformat_to(out, fmt, make_wformat_args(args...));

```

```

template<class Out>
    Out vformat_to(Out out, string_view fmt,
                    format_args_t<type_identity_t<Out>, char> args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt,
                    format_args_t<type_identity_t<Out>, wchar_t> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                    format_args_t<type_identity_t<Out>, char> args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                    format_args_t<type_identity_t<Out>, wchar_t> args);

```

```

template<class Out>
    Out vformat_to(Out out, string_view fmt, format_args args);
template<class Out>
    Out vformat_to(Out out, wstring_view fmt, wformat_args args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                    format_args args);
template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                    wformat_args args);

```

§ 6. Acknowledgements

Thanks to Hana Dušíková for demonstrating that the optimal formatting API can be implemented with P1221.

§ References

§ Informative References

[D-FORMAT]

D Library Reference, `std.format`. URL: https://dlang.org/phobos/std_format.html

[FMT]

Victor Zverovich; et al. [The {fmt} library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[N4861]

Richard Smith; et al. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4861.pdf). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4861.pdf>

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645). URL: <https://wg21.link/p0645>

[P1045]

David Stone. [constexpr Function Parameters](http://wg21.link/p1045). URL: <http://wg21.link/p1045>

[P1221]

Jason Rice. [Parametric Expressions](http://wg21.link/p1221). URL: <http://wg21.link/p1221>

[RUST-FMT]

[The Rust Standard Library, Module `std::fmt`](https://doc.rust-lang.org/std/fmt/). URL: <https://doc.rust-lang.org/std/fmt/>